

ANZ Senior Frontend Engineer - Mock Interview Q&A Cheatsheet

Mục đích: Ôn nhanh trước phỏng vấn. Mỗi câu = Vietnamese hiểu nhanh + English trả lời

Format: 🇻🇳 Hiểu → 🇬🇧 Answer (2-3 câu ngắn gọn, đủ ý)

PART 1: JAVASCRIPT (10-12 câu)

JS-01: var vs let vs const

🇻🇳 `var` = function scope, hoisted (undefined). `let / const` = block scope, hoisted nhưng TDZ (ReferenceError). `const` không reassign được nhưng mutate object OK.



"var is function-scoped and hoisted with undefined. let and const are block-scoped — they're hoisted too, but accessing them before declaration throws a ReferenceError because of the Temporal Dead Zone. const prevents reassignment but allows mutation of objects and arrays. I use const by default, let when reassignment is needed, and never var."

JS-02: Hoisting

🇻🇳 JS "đưa" declarations lên đầu scope. `var` → undefined. `let/const` → TDZ. Function declarations hoisted hoàn toàn, function expressions thì không.



"Hoisting moves declarations to the top of their scope during compilation. var is hoisted with undefined. Function declarations are fully hoisted — callable before they appear. let/const enter

a Temporal Dead Zone until their declaration line. Function expressions are not hoisted — the variable is, but not the assignment."

JS-03: Closure

🇻🇳 Function "nhớ" biến từ scope bên ngoài, kể cả khi scope đó đã chạy xong. Dùng cho: data privacy, function factory, callbacks.



"A closure is when a function retains access to variables from its outer lexical scope, even after that outer function has finished executing. Use cases include data privacy — encapsulating variables, function factories — creating specialized functions, and maintaining context in callbacks. In React, hooks like useState use closures internally."

JS-04: this keyword

🇻🇳 `this` phụ thuộc cách function được gọi: method → object, regular → global/undefined, arrow → lexical (kế thừa từ scope ngoài), new → object mới, call/apply/bind → chỉ định.



"The value of 'this' depends on how the function is called: in a method call, it's the calling object; in a regular function, it's global or undefined in strict mode; arrow functions inherit 'this' lexically from their enclosing scope; with 'new', it's the newly created object; and call/apply/bind can set it explicitly. Arrow functions solve the common 'lost this' problem in callbacks."

JS-05: Event Loop

🇻🇳 JS = single-threaded. Sync code → call stack. Async → queues. **Microtask** (Promise.then) ưu tiên cao hơn **Macrotask** (setTimeout). Sau mỗi task, tất cả microtasks chạy trước khi macrotask tiếp theo.



"JavaScript is single-threaded. The event loop enables async operations: synchronous code runs on the call stack first. When the stack is empty, the event loop checks the microtask queue — Promises, queueMicrotask — which has higher priority. After all microtasks complete, it processes one macrotask — setTimeout, setInterval. This is why Promise.then always resolves before setTimeout, even with delay 0."

```
console.log('A'); // Sync → A
setTimeout(() => console.log('B'), 0); // Macrotask
Promise.resolve().then(() => console.log('C')); // Microtask
console.log('D'); // Sync → D
// Output: A, D, C, B
```

JS-06: Promises & async/await

🇻🇳 Promise = 3 states: pending/fulfilled/rejected. async/await = syntax sugar cho Promise, dễ đọc hơn .then chain. Error handling: try/catch. Parallel: Promise.all() .



"A Promise represents a value that may not be available yet — it has three states: pending, fulfilled, rejected. async/await is syntactic sugar over Promises, making async code look synchronous. I use try/catch for error handling and Promise.all for parallel operations. Promise.allSettled is useful when I want all results regardless of failures."

JS-07: == vs ===

🇻🇳 == type coercion (chuyển type rồi so). === so value + type. Luôn dùng === .



"Double equals performs type coercion before comparison — '1' == 1 is true. Triple equals compares both value and type without coercion — '1' === 1 is false. I always use strict equality because it's predictable. The only edge case where == might be useful is checking null or undefined together, but I prefer being explicit."

JS-08: Prototypal Inheritance

🇻🇳 Mỗi object có `__proto__` link tới prototype của nó. Khi access property, JS tìm từ object → prototype → prototype chain.



"JavaScript uses prototypal inheritance — objects have an internal prototype link. When accessing a property, the engine looks up the prototype chain until it finds the property or reaches null. ES6 classes are syntactic sugar over this mechanism. `Object.create` allows creating objects with a specific prototype without using constructors."

JS-09: Spread/Rest, Destructuring

🇻🇳 Spread (`...`) = trải array/object ra. Rest = gom phần còn lại. Destructuring = tách biến từ object/array.



"Spread expands iterables — useful for copying arrays, merging objects, and passing arguments. Rest collects remaining elements into an array or object. Destructuring extracts values from arrays or properties from objects into distinct variables. These are essential for clean, readable code — I use them extensively in React props and state management."

JS-10: Map, Set, WeakMap, WeakRef

🇻🇳 Map = key-value (key bất kỳ type). Set = giá trị unique. WeakMap/WeakSet = key phải là object, GC tự dọn khi object không còn reference.



"Map stores key-value pairs where keys can be any type — unlike objects which coerce keys to strings. Set stores unique values. WeakMap and WeakSet hold weak references — entries are garbage collected when the key object has no other references. I use Map for lookup tables, Set for deduplication, and WeakMap for attaching metadata to objects without memory leaks."

JS-11: Debounce vs Throttle

🇻🇳 **Debounce**: đợi user ngừng action N ms rồi mới gọi (search input). **Throttle**: chỉ gọi tối đa 1 lần mỗi N ms (scroll, resize).



"Debounce delays execution until after a period of inactivity — ideal for search inputs where you wait for the user to stop typing. Throttle limits execution to at most once per interval — ideal for scroll handlers or resize events. In React, I implement debounce as a custom hook using `useEffect` with a timer cleanup."

JS-12: Deep Copy vs Shallow Copy

🇻🇳 Shallow copy = copy reference (spread, `Object.assign`). Deep copy = copy toàn bộ nested (`structuredClone` , `JSON.parse/stringify`).



"Shallow copy duplicates the top-level properties but nested objects still share references — spread operator and `Object.assign` do this. Deep copy duplicates everything recursively. I use `structuredClone` for deep copies in modern environments, or a utility like `lodash cloneDeep`. `JSON.parse(JSON.stringify())` works but loses functions, dates, and undefined."

PART 2: TYPESCRIPT (8-10 câu)

TS-01: type vs interface

🇻🇳 `interface` : extends, implements, declaration merging. `type` : union, intersection, mapped types, conditional types. Interface cho object shape/API, type cho complex types.



"Interface supports declaration merging and clearer extends syntax — great for object shapes and public APIs. Type is more versatile — it handles unions, intersections, mapped types, and conditional types. My convention: interface for component props and API contracts, type for unions, utility types, and computed types."

TS-02: Generics

🇻🇳 Type parameters để viết code linh hoạt mà vẫn type-safe. Constraint: `T extends HasId`. Thực tế: API response wrapper, generic components.



"Generics let you write reusable, type-safe code without losing type information. Instead of 'any', a generic function like `getFirst<T>(arr: T[]): T` preserves the actual type. Constraints with 'extends' limit acceptable types. In React, I use generics for reusable components like `List<T>` and for API utilities like `ApiResponse<T>` — this provides autocompletion and compile-time safety."

```
interface ApiResponse<T> {  
  data: T;  
  status: number;  
}  
  
async function fetchApi<T>(url: string): Promise<ApiResponse<T>> {  
  const res = await fetch(url);  
  return res.json();  
}  
  
const users = await fetchApi<User[]>('/api/users'); // data is User[]
```

TS-03: Utility Types

🇻🇳 `Partial<T>` = all optional. `Pick<T,K>` = chọn fields. `Omit<T,K>` = bỏ fields. `Record<K,V>` = object type. `ReturnType<T>` = return type của function. `Required<T>` = all required.



"The utility types I use most: Partial for update DTOs where any field can change. Pick and Omit to create type subsets — Omit is great for removing internal fields from API responses. Record for dictionaries and lookup objects. ReturnType to infer return types of functions I don't control. These avoid type duplication and keep types in sync as the codebase evolves."

TS-04: Type Guards

🇻🇳 Giúp TypeScript hiểu type cụ thể lúc runtime: `typeof` (primitive), `instanceof` (class), `in` (property check), custom `user is Admin`.



"Type guards narrow types at runtime. Built-in options: `typeof` for primitives, `instanceof` for classes, `'in'` operator for property checks. For custom narrowing, I use type predicates with the `'is'` keyword — `function isAdmin(user): user is Admin`. This is essential for discriminated unions and API responses where shape varies by status. If I can't narrow without assertion, it usually means the types need redesigning."

TS-05: Discriminated Unions

🇻🇳 Union type với 1 property chung (discriminant) để phân biệt. Switch/if trên discriminant → TS tự narrow type.



"Discriminated unions use a common property — the discriminant — with unique literal values to differentiate members. TypeScript automatically narrows the type when you check the discriminant in a switch statement. I use this for API state management — Loading, Success, Error types with a status field — and for Redux-style actions. It ensures exhaustive handling — if I add a new variant, TypeScript errors where I need to handle it."

```
type ApiState =  
  | { status: 'loading' }  
  | { status: 'success'; data: User[] }  
  | { status: 'error'; error: string };
```

TS-06: React + TypeScript Patterns

🇻🇳 Props = interface. Event = `React.ChangeEvent<HTMLInputElement>` . Children = `React.ReactNode` . `forwardRef` = `forwardRef<HTMLInputElement, Props>` . Generic component = `function List<T>(props: ListProps<T>)` .



"I define props as interfaces with clear types. For events, React provides typed handlers like `ChangeEvent`, `MouseEvent`, `FormEvent` — each generic over the element type. Children are typed as `ReactNode` for flexibility. For `forwardRef`, the generic order is `forwardRef<RefType, PropsType>` . For reusable components, I use generic props like `SelectProps<T>` so the component works with any data type while maintaining full type safety. I avoid 'any' and prefer 'unknown' with type guards."

TS-07: Conditional Types & Mapped Types

🇻🇳 Conditional: `T extends string ? A : B` . Mapped: transform properties `{ [K in keyof T]: ... }` . Dùng cho advanced utility types.



"Conditional types work like ternary operators at the type level — `T extends string ? StringHandler : DefaultHandler` . Mapped types transform properties of existing types — the built-in `Partial`, `Required`, and `Readonly` are all mapped types. I use them for creating API-specific type transformations, like making all `Date` fields into strings for serialization, or making specific fields optional for update operations."

TS-08: 'unknown' vs 'any' vs 'never'

🇻🇳 `any` = tắt type checking. `unknown` = safe any, phải narrow trước khi dùng. `never` = không bao giờ xảy ra (exhaustive check).



"any disables type checking entirely — I avoid it. unknown is the type-safe counterpart: you must narrow before using it, which forces runtime checks. never represents values that should never occur — useful for exhaustive switch statements. If I add a new union member and forget to handle it, a never check in the default case will cause a compile error. This is essential for maintaining type safety at scale."

PART 3: REACT (15-18 câu)

R-01: Virtual DOM & Reconciliation

🇻🇳 Virtual DOM = bản copy nhẹ của Real DOM. Khi state đổi → tạo VDOM mới → diff với cũ → update chỉ phần thay đổi. Diffing $O(n)$ nhờ 2 giả định: element khác type = tree khác, keys giúp identify items.



"The Virtual DOM is a lightweight JavaScript representation of the actual DOM. When state changes, React creates a new VDOM tree, diffs it against the previous one — this is called reconciliation — and calculates the minimal DOM updates needed. The diffing algorithm runs in $O(n)$ by assuming elements of different types produce different trees, and keys help identify which items changed in lists. This batch updating is much more efficient than direct DOM manipulation."

R-02: Hooks Overview

🇻🇳 `useState` = local state. `useEffect` = side effects. `useCallback` = cache function ref. `useMemo` = cache value. `useRef` = mutable value không trigger re-render. `useContext` = consume context. `useReducer` = complex state logic.



"useState manages local state with optional functional updates for state depending on previous values. useEffect handles side effects with a dependency array controlling when it runs — empty

array for mount only. useCallback memoizes function references for stable callbacks passed to memoized children. useMemo caches expensive computed values. useRef stores mutable values that persist across renders without triggering re-renders — useful for DOM access and timer IDs. useReducer centralizes complex state transitions."

R-03: useState - Functional Updates & Batching

🇻🇳 `setCount(prev => prev + 1)` khi state mới phụ thuộc state cũ. React 18 batches tất cả state updates (kể cả trong `setTimeout/Promise`).



"Functional updates — `setState(prev => prev + 1)` — are essential when the new state depends on the previous value. Without them, multiple calls in the same handler see the same stale state due to closures. React 18 automatically batches all state updates, even inside Promises and `setTimeout`, which reduces unnecessary re-renders. Lazy initialization — `useState(() => expensiveComputation())` — runs only on mount."

R-04: useEffect - Deps, Cleanup, Common Patterns

🇻🇳 Deps array: không có = every render, `[]` = mount only, `[a,b]` = khi a/b đổi. Cleanup chạy trước next effect và trước unmount. Pattern: cancelled flag cho fetch, remove listener.



"The dependency array controls when effects run: no array means every render, empty array means mount only, and specific deps means when those values change. The cleanup function runs before the next effect execution and on unmount. For data fetching, I use a cancelled flag to prevent state updates after unmount. For event listeners, I add in setup and remove in cleanup. The exhaustive-deps ESLint rule helps catch missing dependencies that cause stale closures."

R-05: useMemo vs useCallback

🇻🇳 `useMemo` = cache giá trị tính toán. `useCallback` = cache function reference. Chỉ dùng khi: tính toán nặng, truyền xuống `React.memo` child, hoặc là dep của hook khác. **Đừng lạm dụng** — overhead có thể lớn hơn benefit.



"useMemo caches a computed value — for expensive calculations like filtering large datasets. useCallback caches a function reference — useful when passing callbacks to children wrapped in `React.memo`. However, I don't use them everywhere. If the computation is cheap, `useMemo`'s overhead outweighs the benefit. `useCallback` is pointless if the child isn't memoized. I measure before optimizing — premature optimization makes code harder to read without measurable gains."

R-06: useRef - DOM Access & Mutable Values

🇻🇳 `.current` = mutable, persist qua renders, không trigger re-render. Dùng cho: DOM ref, timer ID, previous value, store mutable data.



"useRef returns a mutable object persisting across renders without causing re-renders. Primary uses: DOM access for focusing inputs or measuring elements; storing timer/interval IDs for cleanup; tracking previous values with a `usePrevious` pattern; and holding any mutable value that shouldn't affect rendering. The key difference from state: changing `ref.current` is synchronous and doesn't trigger a re-render."

R-07: Context API & When NOT to Use It

🇻🇳 Context = share data qua component tree không cần prop drilling. **Hạn chế:** mọi consumer re-render khi value đổi → không tốt cho high-frequency updates.



"Context is great for infrequently changing data shared across many components — theme, auth, locale. However, it has a key limitation: every consumer re-renders when the context value changes, regardless of which part they use. This makes it unsuitable for high-frequency updates like form state or real-time data. For those cases, I use Zustand or Jotai for global state, and React Query for server state. Context is a tool, not a state management solution."

R-08: Custom Hooks

🇻🇳 Extract + reuse stateful logic. Mỗi component dùng hook = state riêng biệt (share logic, không share state). Bắt đầu "use".



"Custom hooks extract and reuse stateful logic between components. They follow the same rules as built-in hooks — must start with 'use', called at top level. Each component using the hook gets its own isolated state — they share logic, not state. I commonly create useDebounce, useLocalStorage, useMediaQuery, useOnClickOutside, and useFetch. The power is composability — combine multiple hooks into higher-level abstractions."

R-09: React.memo & Performance

🇻🇳 `React.memo` = skip re-render nếu props không đổi. Kết hợp với `useCallback` / `useMemo` để đảm bảo props stable. Custom comparator cho control chi tiết.



"React.memo is a higher-order component that skips re-rendering when props haven't changed, using shallow comparison by default. It's effective for expensive components rendered frequently. But it requires stable props — combine with `useCallback` for function props and `useMemo` for object props. For fine-grained control, pass a custom comparison function. I only use it when I've identified actual performance issues with React DevTools Profiler."

R-10: Code Splitting & Lazy Loading

🇻🇳 `React.lazy` + `Suspense` = tải component khi cần. Route-based splitting phổ biến nhất. Giảm initial bundle size.



"Code splitting with `React.lazy` and `Suspense` loads components on demand, reducing the initial bundle. I primarily use route-based splitting — each route loads its own chunk. For heavy features within a page, I use component-level splitting. `Suspense` provides fallback UI during loading. In `Next.js`, this is mostly automatic with the App Router. For named exports, I use `lazy(() => import('./Module').then(m => ({ default: m.Component })))`."

R-11: State Management Strategy

🇻🇳 Local (`useState`) → Lifted (props) → Context (theme/auth) → Global (`Zustand`) → Server (`React Query`) → URL (router).



"I categorize state by nature: local state with `useState` for component-specific data. Server state with `React Query` — it handles caching, background refetching, and optimistic updates. Global UI state with `Zustand` for lightweight, low-boilerplate global state. URL state for anything shareable or bookmarkable — filters, pagination. Context only for infrequently changing data like theme or locale. This separation keeps each layer simple and testable."

R-12: React Patterns (Custom Hooks vs HOC vs Render Props)

🇻🇳 Custom Hooks = modern, preferred. HOC = `function(Component) → enhanced Component`, dùng cho auth wrapper. Render Props = component nhận function as prop. Hooks thay thế hầu hết use cases của HOC/Render Props.



"Custom Hooks are my default for sharing stateful logic — they're composable, TypeScript-friendly, and don't add wrapper components. HOCs wrap components to add behavior like authentication — still useful but cause wrapper hell and unclear prop origins. Render Props pass behavior through function props — useful for UI composition. In modern React, hooks replace most HOC and render prop use cases. I use HOCs occasionally for cross-cutting concerns like route protection."

R-13: Compound Components Pattern

🇻🇳 Components chia sẻ implicit state qua Context. VD: `<Select>` + `<Select.Option>` . Thay vì 15 props → composition linh hoạt.



"Compound components provide a flexible API where a parent shares implicit state with children through Context. Instead of a single component with 15 props, I split into composable parts — `Modal.Header`, `Modal.Body`, `Modal.Footer`. The parent manages state, children consume it. Benefits: flexible composition, readable JSX, no prop drilling. Libraries like Radix UI and Headless UI use this pattern extensively. I apply it for Tabs, Accordions, Dropdowns, and Modals."

R-14: Controlled vs Uncontrolled Components

🇻🇳 Controlled = React state là source of truth (value + onChange). Uncontrolled = DOM quản lý (defaultValue + ref). Controlled preferred vì explicit data flow.



"Controlled components use React state as the single source of truth — value is set by state and updated via onChange. This enables real-time validation, conditional formatting, and full control over form data. Uncontrolled components let the DOM manage the value — accessed via refs on submission. I prefer controlled for most cases because the data flow is explicit and debuggable. Uncontrolled is useful for file inputs or integrating with non-React libraries."

R-15: Error Boundaries

🇻🇳 Class component duy nhất bắt render errors. `getDerivedStateFromError` + `componentDidCatch` . Dùng ở route level và feature level.



"Error Boundaries are class components that catch JavaScript errors in their child component tree during rendering. They use `getDerivedStateFromError` to render fallback UI and `componentDidCatch` to log errors. I place them at route level for page-wide errors and around critical features for granular recovery. They don't catch errors in event handlers, async code, or server-side rendering — those need `try/catch`. React 19 is adding functional error boundary support."

R-16: Server Components vs Client Components (Next.js)

🇻🇳 Server Component (default) = render trên server, có thể `async/await`, query DB. Client Component = `'use client'` , dùng hooks, events. Strategy: bắt đầu Server, thêm `'use client'` khi cần interactivity.



"In Next.js App Router, components are Server Components by default — they can `async/await`, access databases directly, and their code never ships to the client, reducing bundle size. Client Components need the `'use client'` directive — required for hooks, event handlers, and browser APIs. My strategy: start with Server Components, push `'use client'` boundary as far down as possible. This gives the best performance — only interactive parts are hydrated on the client."

R-17: Data Fetching Patterns (Next.js)

🇻🇳 Default = cached. `revalidate: 3600` = ISR. `cache: 'no-store'` = dynamic. Server Actions = form submit trực tiếp từ server. `Promise.all` cho parallel fetch.



"Next.js 14 provides several fetching strategies: default fetch is cached for static data. Adding `next: { revalidate: 3600 }` enables ISR — pages revalidate after the specified interval. `cache: 'no-store'` ensures fresh data every request for dynamic content. Server Actions with 'use server' handle form submissions without API routes — they even work with JavaScript disabled. For parallel data fetching, I use `Promise.all` to avoid waterfall requests."

R-18: React Performance Optimization Checklist

🇻🇳 React.memo, useMemo, useCallback, code splitting, virtualization (react-window), image lazy loading, avoid inline objects/functions, React DevTools Profiler.



"My performance optimization approach: first, measure with React DevTools Profiler to identify actual bottlenecks. Then apply targeted optimizations: React.memo for expensive pure components, useMemo/useCallback for stable references, code splitting for reduced initial bundle, virtualization with react-window for long lists, image lazy loading with blur placeholders. I avoid premature optimization — three similar lines of code is better than a premature abstraction. Always measure the impact after each optimization."

PART 4: UI / DESIGN SYSTEM (6-8 câu)

UI-01: Component Layers

🇻🇳 4 tầng: Design Tokens (màu, spacing) → Primitives (Button, Input) → Composed (SearchInput, DataTable) → Feature (LoginForm).



"I organize components in layers: Layer 0 — Design Tokens define colors, spacing, typography as variables for consistency and theming. Layer 1 — Primitives like Button, Input are pure UI with no business logic, shareable across projects. Layer 2 — Composed components like SearchInput combine primitives, reusable across features. Layer 3 — Feature components like

LoginForm contain domain-specific logic. Primitives are the most reusable, feature components are the most specific."

UI-02: Design Tokens

🇻🇳 Biến CSS có ý nghĩa: `--color-primary` , `--spacing-md` . Thay vì hardcode `#3b82f6` khắp nơi → đổi 1 chỗ apply everywhere. Enable theming (dark mode).



"Design tokens are the single source of truth for design values — colors, spacing, typography, shadows. Instead of hardcoding hex values, I define semantic tokens like 'color-primary' or 'spacing-md'. Benefits: change once to apply everywhere, easy theming by swapping token sets for dark mode, consistent language between design and engineering. I implement them as CSS custom properties for runtime theming, or as Tailwind config values."

UI-03: Wrapper Components for Libraries

🇻🇳 Không import MUI/Antd trực tiếp khắp nơi. Tạo wrapper → tất cả import từ wrapper → đổi library = sửa 1 file.



"I always wrap third-party UI components. Instead of importing MUI Button directly in 100 files, I create a Button wrapper in my components folder. This provides: a single point of change when switching libraries, a consistent API that my team controls, and type safety with exactly the props we support. In a recent project, we migrated from MUI to Shadcn. Because of wrappers, it took days instead of weeks."

UI-04: MUI vs Antd vs Shadcn/ui

🇻🇳 MUI = Material Design, nặng, khó customize. Antd = enterprise, rất nặng. **Shadcn/ui** = copy code vào project, dùng Radix UI + Tailwind, full control, nhẹ.



"MUI is fast to start with Material Design but heavy and hard to customize deeply. Ant Design is comprehensive for enterprise apps but very opinionated. Shadcn/ui is my preference for new projects — it's not an npm dependency; you copy components into your codebase, built on Radix UI for accessibility and styled with Tailwind. You get full control, smaller bundles since you only include what you use, and easy customization since the code is yours."

UI-05: Accessibility (WCAG AA)

★ Contrast 4.5:1 (text), 3:1 (large text). Keyboard: tất cả interactive focusable, tab order logic, focus indicator rõ. Forms: label + input linked, error rõ ràng. ARIA: dialog, alert, live regions.



"Key WCAG AA requirements: color contrast at least 4.5:1 for normal text; all interactive elements keyboard-focusable with visible focus indicators; form inputs linked to labels with clear error messages using aria-describedby; semantic HTML with proper heading hierarchy; ARIA roles for custom widgets — dialog, alert, combobox. For testing, I use axe-core in CI, manual keyboard testing, and VoiceOver. Using getByRole in Testing Library also validates accessibility automatically."

UI-06: Responsive Design Approach

★ Mobile-first. Breakpoints: mobile → tablet → desktop. CSS: media queries / Tailwind responsive prefixes. Layout: Flexbox/Grid. Testing: Chrome DevTools device mode.



"I follow mobile-first responsive design — start with mobile layout, enhance for larger screens. I use Tailwind's responsive prefixes (sm, md, lg, xl) which maps to standard breakpoints. For layout, CSS Grid handles two-dimensional layouts while Flexbox handles one-dimensional alignment. I use the useMediaQuery custom hook for JavaScript-dependent responsive behavior. Testing across devices is essential — I use Chrome DevTools device mode and real device testing for critical flows."

UI-07: Storybook

🇻🇳 Playground develop + document components. Develop isolated, auto-docs từ TS props, visual testing all states, team review.



"Storybook is essential for component development: isolated development without running the full app, auto-generated documentation from TypeScript props, visual testing of all states — loading, error, empty, with data. We deploy it as a static site for team collaboration — designers review components directly, and new team members use it to understand available components. It also enables visual regression testing when combined with Chromatic."

UI-08: Design Modal System

🇻🇳 Portal (`createPortal` vào body). Focus trap (Tab chỉ trong modal). Escape close. Restore focus khi đóng. Body scroll lock. `role="dialog" , aria-modal="true" .`



"For a Modal system: I render with `createPortal` to `document.body` to avoid z-index issues. Accessibility is critical — `role='dialog'` with `aria-modal='true'`, `aria-labelledby` pointing to the title. Focus trap intercepts Tab to keep focus inside the modal. Escape key closes it. When the modal closes, focus returns to the trigger element. Body scroll is locked with `overflow:hidden`. I use the compound component pattern — `Modal.Header`, `Modal.Body`, `Modal.Footer` — for flexible composition."

PART 5: TESTING (5-6 câu)

T-01: Testing Strategy (Testing Trophy)

🇻🇳 Mostly integration tests > unit tests > few E2E. Integration tests cho confidence cao nhất mà cost vừa phải.



"I follow the Testing Trophy approach: mostly integration tests because they give the highest confidence that components work together correctly. Unit tests for complex pure logic — utility functions, custom hooks, reducers. A few E2E tests with Playwright for critical user journeys — login, checkout, payment. This balance maximizes confidence per test dollar. I avoid testing implementation details — test behavior, not how it's implemented."

T-02: React Testing Library Philosophy

🇻🇳 "Test like a user." Query ưu tiên: getByRole > getByLabelText > getByText > getByTestId. Nếu không tìm được bằng role → a11y có vấn đề.



"Testing Library's philosophy is testing how users interact with your app, not implementation details. Query priority: getByRole first — it tests accessibility automatically; getByLabelText for form inputs; getByText for displayed content; getByTestId only as last resort. If I can't find an element by role, it often means the accessibility is broken. I use userEvent over fireEvent for more realistic interaction simulation."

T-03: MSW (Mock Service Worker)

🇻🇳 Mock API ở tầng network (không mock fetch). Realistic hơn, cùng mocks dùng cho test + dev. Define handlers cho success, override cho error tests.



"I use MSW to intercept requests at the network level rather than mocking fetch directly. This is more realistic — actual fetch calls are made. Same mock handlers work in tests and browser development. I define success handlers as defaults, then override them in specific tests for error

scenarios. This approach tests actual data fetching behavior, including loading states, error handling, and data transformation — not just that a mock function was called."

T-04: Testing Custom Hooks

🇻🇳 renderHook từ Testing Library. `jest.useFakeTimers()` cho timer-based hooks. `act()` wrap state updates. `rerender()` để test value changes.



"I test custom hooks using renderHook from Testing Library. For timer-based hooks like `useDebounce`, I use `jest.useFakeTimers` and `advanceTimersByTime`. State updates are wrapped in `act()` to ensure React processes them. The `rerender` function tests how hooks respond to prop changes. For hooks with side effects, I verify cleanup runs correctly. This approach tests the hook's contract without coupling to a specific component."

T-05: Testing Async Components

🇻🇳 `waitFor` để đợi async updates. `findBy` = `waitFor` + `getBy`. Test loading → success flow. Override MSW handler cho error flow.



"For async components: I render, verify the loading state appears, then use `waitFor` or `findBy` queries to wait for the final state. The pattern is: render component, assert loading indicator, await data display. For error testing, I override MSW handlers in that specific test. I always test the full lifecycle: loading → success, and loading → error → retry. Using MSW ensures the async behavior is tested realistically."

T-06: What to Test vs What NOT to Test

🇻🇳 Test: user interactions, conditional rendering, API integration, error handling. Không test: implementation details, library internals, CSS styling.



"I test: user-visible behavior — what renders, what happens on interaction; conditional rendering — does the right UI show for different states; API integration — loading, success, error flows; form validation — required fields, format validation. I don't test: implementation details like state variables; third-party library internals; pure styling changes. The guideline is: if it affects what the user sees or does, test it."

PART 6: OPERATIONS / CI-CD (4-5 câu)

O-01: CI/CD Pipeline

🇻🇳 Push → Lint → Type check → Unit tests → Build → E2E tests → Preview deploy (PR) → Production deploy (merge main).



"My CI/CD pipeline with GitHub Actions includes: lint and type-checking on every push; unit and integration tests with coverage checks; build verification; E2E tests with Playwright for critical flows; preview deployments for every PR so reviewers can test; and auto-deploy to production on merge to main. Key practices: branch protection requiring passing CI, caching node_modules, parallel test execution, and Slack notifications for failures."

O-02: Frontend Monitoring

🇻🇳 Sentry (errors + breadcrumbs), Core Web Vitals (LCP, CLS, INP), Lighthouse CI (budget), Real User Monitoring.



"I implement monitoring at three levels: Error tracking with Sentry — capturing unhandled errors, promise rejections, with release versioning for regression detection and breadcrumbs for context. Performance monitoring with Core Web Vitals — tracking LCP under 2.5s, CLS under

0.1, INP under 200ms for real users. Lighthouse CI in the pipeline with performance budgets that block PRs if thresholds are exceeded. For critical apps, Real User Monitoring with session replays to understand user frustration."

O-03: Incident Handling

🇻🇳 Detect → Assess severity → Mitigate (rollback/feature flag) → Communicate → Resolve → Post-mortem.



"My incident response follows: Detect via alerts from Sentry or monitoring dashboards. Assess severity — P0 is site down, P1 is major feature broken. Mitigate quickly — either rollback deployment, disable via feature flag, or deploy hotfix. Communicate status to stakeholders every 15 minutes. After resolution, conduct a blameless post-mortem — document root cause, timeline, and preventive actions. Feature flags are essential for instant mitigation without deployment."

O-04: Feature Flags

🇻🇳 Gradual rollout (5% → 50% → 100%). Kill switch khi có bug. A/B testing. Beta features cho specific users.



"Feature flags enable safe deployments: gradual rollout — start with 5% of users, monitor error rates and metrics, then increase gradually. Kill switch — every major feature has a flag for instant disable without deployment. A/B testing — compare two implementations with real users. Beta features — enable for internal users or specific accounts for testing. This lets us merge to main continuously and revert features without code deploys."

O-05: Core Web Vitals

🇻🇳 LCP (Largest Contentful Paint) < 2.5s. CLS (Cumulative Layout Shift) < 0.1. INP (Interaction to Next Paint) < 200ms.



"Core Web Vitals measure real user experience: LCP measures loading — the largest content element should render within 2.5 seconds. CLS measures visual stability — layout shouldn't shift unexpectedly, target under 0.1. INP replaced FID — it measures responsiveness to all interactions, target under 200ms. I track these with the web-vitals library, set Lighthouse CI budgets in the pipeline, and monitor trends in production dashboards."

PART 7: SYSTEM DESIGN (Framework)

SD-01: RADIO Framework

🇻🇳 Requirements → **Architecture** → **Data Model** → **Interface** → **Optimizations**



"I use the RADIO framework for frontend system design:

Requirements (5 min): Clarify functional and non-functional — scale, performance targets, device support, accessibility, offline needs.

Architecture (10 min): Component hierarchy with containers and presenters, state layer (local, global, server), service layer (API, WebSocket, storage).

Data Model (5 min): TypeScript interfaces for entities, API contracts with request/response shapes, caching strategy.

Interface (5 min): Component props and events, composition patterns, data flow direction.

Optimizations (5 min): Performance (code splitting, virtualization), accessibility (ARIA, keyboard), edge cases (empty, error, loading states)."

SD-02: State Management Architecture

🇻🇳 Tách state theo loại: local (useState) → global UI (Zustand) → server (React Query) → URL (router). Không dùng 1 tool cho tất cả.



"I separate state by concern: local state with useState for component-specific data; server state with React Query for API data with caching and background refetching; global UI state with Zustand for app-wide UI concerns; URL state for shareable data like filters and pagination. This separation prevents a single bloated store and makes each layer independently testable. The key insight is that server state has different needs than UI state — React Query handles staleness, caching, and synchronization that a generic store can't."

SD-03: API Communication

🇻🇳 REST = default, HTTP caching dễ. GraphQL = complex data, giảm over-fetching. Real-time: Polling (simple) → SSE (server push) → WebSocket (bi-directional).



"REST is my default for CRUD operations — well-understood, great HTTP caching support, works well with React Query. GraphQL when data requirements are complex or multiple clients need different data shapes — eliminates over-fetching and reduces round trips. For real-time: polling for simple low-frequency checks; Server-Sent Events for one-way server pushes like live feeds; WebSockets for bidirectional high-frequency communication like chat or trading. I choose based on the specific data flow requirements."

PART 8: EXPERIENCE / BEHAVIORAL (Quick STAR)

E-01: "Tell me about yourself" (2 min)



"I'm a Senior Frontend Engineer with 5+ years of experience in React and TypeScript. Currently at Cognisian, I lead the frontend for a healthcare information system — a monorepo with 5 applications serving patients, doctors, and administrators. Before that, I built a real-time trading platform at Bolt Technologies from scratch, handling 30K+ records with Web Workers and WebSocket optimization. I also worked on a multi-tenant booking platform at Eye Design in Sydney, collaborating across AU and VN timezones. I'm passionate about architecture decisions, developer experience, and mentoring junior developers."

E-02: Architecture Decision (Monorepo)



S: At Cognisian, 5 separate repos caused inconsistent UI, duplicate code, and version mismatches.

T: As Frontend Lead, I needed to improve consistency and developer experience.

A: Migrated to a Turbo monorepo with shared packages — UI components, utilities, and type-safe API client.

R: Build times reduced 60%, seamless code sharing, faster onboarding for new team members."

E-03: Performance Problem (Trading Platform)



S: Trading platform loading 30K+ records caused 8-second load times and UI freezing.

T: Make it responsive enough for professional traders expecting sub-second performance.

A: Implemented Web Workers for heavy calculations, virtualization for the order book, data chunking with progressive loading, and WebSocket update batching.

R: Load time dropped from 8s to 2s, UI maintained 60fps during market volatility, zero performance complaints."

E-04: Mentoring



S: Two junior developers joined the team with basic React skills.

T: Get them contributing effectively within 2 months.

A: Structured mentoring: weekly 1:1s, pair programming, internal documentation on our patterns, progressively complex task assignments, and educational code reviews.

R: Both productive within 6 weeks, one now leads features independently, created reusable onboarding materials."

E-05: Cross-Geography Collaboration



S: At Eye Design, team distributed between Sydney and Vietnam with 7-hour timezone gap.

T: Ensure smooth collaboration and timely delivery.

A: Async-first communication with detailed PRs and Loom videos, overlap hours for real-time discussions, documentation culture with ADRs, and clear feature ownership.

R: Reduced back-and-forth by 50%, same-day client feedback turnaround, successfully launched in both markets."

E-06: Production Incident



S: Trading platform order book stopped updating in real-time — P0 for traders.

T: Identify and fix while keeping stakeholders informed.

A: Checked Sentry, found WebSocket reconnection errors. Deployed feature flag fallback to polling as mitigation. Fixed the reconnection bug with proper cleanup logic.

R: Mitigated in 20 minutes, fully resolved in 2 hours. Post-mortem led to WebSocket health monitoring and automatic fallback to polling."

E-07: Questions to Ask Interviewer



- "What does success look like in this role after 6 months?"
- "What are the biggest technical challenges the team is facing?"
- "How does the team balance new features vs technical debt?"
- "What's the testing and deployment process like?"
- "How is the team structured — cross-functional or specialized?"

NEXT.JS 14 DEEP DIVE (10-12 câu)

NX-01: App Router vs Pages Router

🇻🇳 App Router (Next 13.4+) dùng thư mục `app/` với React Server Components mặc định, nested layouts, streaming SSR. Pages Router dùng `pages/` với `getServerSideProps` / `getStaticProps`. Khi migrate, ưu tiên chuyển routes ít phụ thuộc trước, 2 router chạy song song được.



"The App Router is the new paradigm introduced in Next.js 13.4 — it uses the `app/` directory with React Server Components by default, nested layouts, and built-in streaming. The Pages Router uses `pages/` with data fetching via `getServerSideProps` and `getStaticProps`. A key advantage of App Router is that layouts don't re-render on navigation, which gives you a smoother SPA-like feel. For migration, both routers can coexist — so at ANZ I'd recommend an incremental approach: migrate leaf routes first, then work inward toward shared layouts, validating each step in staging."

Pages Router:

pages/

- |— index.tsx → /
- |— about.tsx → /about
- |— _app.tsx → wrapper
- |— api/hello.ts → API
- |— _document.tsx → HTML shell

App Router:

app/

- |— page.tsx → /
- |— about/page.tsx → /about
- |— layout.tsx → nested layout
- |— api/hello/route.ts → Route Handler
- |— loading.tsx → Suspense boundary

NX-02: Server Components vs Client Components

🇻🇳 Server Components (mặc định trong App Router) render trên server, không gửi JS về client, truy cập DB/API trực tiếp. Client Components cần `'use client'` directive, dùng khi cần hooks, event handlers, browser APIs. Chiến lược: đẩy `'use client'` xuống thấp nhất có thể (leaf components).



"In the App Router, every component is a Server Component by default — it renders on the server, sends zero JavaScript to the client, and can directly access databases or internal APIs. When you need interactivity — `useState`, `onClick`, browser APIs — you add `'use client'` at the top of the file to make it a Client Component. My strategy is to push the `'use client'` boundary as low as possible in the component tree. For example, a product page stays as a Server Component, but only the 'Add to Cart' button is a Client Component. This minimizes the client bundle while keeping the app interactive where needed."

```
// app/products/[id]/page.tsx – Server Component (default)
import { AddToCartButton } from './AddToCartButton';

export default async function ProductPage({ params }: { params: { id: string } }) {
  const product = await db.product.findUnique({ where: { id: params.id } }); // Direct
  return (
    <div>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
      <AddToCartButton productId={product.id} /> {/* Client boundary here */}
    </div>
  );
}

// app/products/[id]/AddToCartButton.tsx – Client Component
'use client';
import { useState } from 'react';
export function AddToCartButton({ productId }: { productId: string }) {
  const [adding, setAdding] = useState(false);
  return <button onClick={() => setAdding(true)}>Add to Cart</button>;
}
```

NX-03: Server Actions

🇺🇸 Server Actions là async functions chạy trên server, gọi trực tiếp từ Client Components qua form hoặc event handler. Đánh dấu bằng 'use server'. Tự động tạo POST endpoint, hỗ trợ progressive enhancement (form hoạt động khi JS tắt). Dùng `revalidatePath` / `revalidateTag` để cập nhật cache sau mutation.



"Server Actions are async functions that execute on the server but can be called directly from the client — you mark them with 'use server'. They're great for form submissions and data mutations because Next.js automatically creates a secure POST endpoint behind the scenes. The big win is progressive enhancement — forms work even with JavaScript disabled, which is important for accessibility. After a mutation, I call `revalidatePath` or `revalidateTag` to bust the cache and show fresh data. At ANZ, this pattern would be ideal for banking forms where reliability and progressive enhancement matter."

```
// app/actions/transfer.ts
'use server';
import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';

export async function submitTransfer(formData: FormData) {
  const amount = formData.get('amount') as string;
  const toAccount = formData.get('toAccount') as string;

  // Server-side validation
  if (Number(amount) <= 0) throw new Error('Invalid amount');

  await db.transfer.create({ data: { amount: Number(amount), toAccount } });

  revalidatePath('/transactions'); // Bust cache
  redirect('/transactions/success');
}

// app/transfer/page.tsx – Works even with JS disabled!
import { submitTransfer } from '../actions/transfer';

export default function TransferPage() {
  return (
    <form action={submitTransfer}>
      <input name="toAccount" required />
      <input name="amount" type="number" required />
      <button type="submit">Transfer</button>
    </form>
  );
}
```

NX-04: Data Fetching Strategies

🇻🇳 Next.js 14 mở rộng `fetch()` với caching: `force-cache` (mặc định, cache vĩnh viễn), `no-store` (luôn fetch mới), `revalidate: N` (ISR theo thời gian). Parallel fetching bằng `Promise.all` để tránh request waterfall. Request Memoization tự dedupe cùng 1 URL trong 1 render pass.



"Next.js 14 extends the native fetch API with powerful caching options. By default, fetch uses force-cache — meaning data is cached indefinitely until you explicitly revalidate. Setting cache: 'no-store' gives you dynamic data on every request, similar to getServerSideProps. For time-based ISR, you use next: { revalidate: 60 }. To avoid request waterfalls, I always use Promise.all for independent data fetches — this is crucial for performance. Next.js also has automatic request memoization, so if multiple components fetch the same URL in one render pass, the request only fires once."

```
// app/dashboard/page.tsx
export default async function Dashboard() {
  // Parallel fetching — no waterfall!
  const [user, transactions, notifications] = await Promise.all([
    fetch('https://api.anz.com/user', { cache: 'no-store' }), // Always fresh
    fetch('https://api.anz.com/transactions', {
      next: { revalidate: 30, tags: ['transactions'] } // ISR: 30s
    }),
    fetch('https://api.anz.com/notifications', { cache: 'no-store' })
  ]).then(responses => Promise.all(responses.map(r => r.json())));

  return (
    <div>
      <UserHeader user={user} />
      <TransactionList data={transactions} />
      <NotificationBell count={notifications.unread} />
    </div>
  );
}
```

NX-05: Route Handlers (API Routes in App Router)

🚩 Route Handlers thay thế pages/api/*, đặt trong app/api/*/route.ts. Export các hàm theo HTTP method: GET, POST, PUT, DELETE. Mặc định GET được cache static, thêm dynamic behavior (cookies, headers, POST) sẽ tự chuyển dynamic. Hỗ trợ streaming responses.



"Route Handlers are the App Router replacement for API routes — you create a route.ts file inside app/api and export named functions for each HTTP method like GET, POST, PUT,

DELETE. By default, GET handlers are statically cached at build time, but they become dynamic when you access cookies, headers, or use non-GET methods. They use the standard Web Request/Response APIs, which makes them portable. I also use them for webhook endpoints and to create BFF patterns that aggregate multiple backend gRPC calls into a single REST response for the frontend."

```
// app/api/accounts/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { cookies } from 'next/headers';

export async function GET(request: NextRequest) {
  const token = cookies().get('session')?.value;
  if (!token) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const accounts = await grpcClient.getAccounts(token);
  return NextResponse.json(accounts);
}

export async function POST(request: NextRequest) {
  const body = await request.json();
  const result = await grpcClient.createAccount(body);
  return NextResponse.json(result, { status: 201 });
}

// app/api/accounts/[id]/route.ts – Dynamic segment
export async function GET(_req: NextRequest, { params }: { params: { id: string } }) {
  const account = await grpcClient.getAccount(params.id);
  return NextResponse.json(account);
}
```

NX-06: Middleware

🚩 Middleware chạy trước MỌI request ở Edge Runtime, dùng file `middleware.ts` ở root. Use cases: auth check (redirect nếu chưa login), i18n (detect locale → redirect), rate limiting, A/B testing, header injection. Config `matcher` để chỉ chạy cho routes cần thiết.



"Next.js Middleware runs before every matched request at the Edge — it's defined in a single `middleware.ts` file at the project root. The primary use cases I've implemented are auth guards — checking tokens and redirecting unauthenticated users, internationalization — detecting locale from headers and redirecting accordingly, and security headers injection. At a bank like ANZ, middleware is critical for enforcing authentication on all protected routes before the page even starts rendering. You use the `matcher` config to scope it — for example, excluding public routes and static assets."

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value;
  const { pathname } = request.nextUrl;

  // Auth guard
  if (!token && pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // i18n: detect locale
  const locale = request.headers.get('accept-language')?.split(',')[0]?.split('-')[0] |
  if (!pathname.startsWith('/en') && !pathname.startsWith('/vi')) {
    return NextResponse.redirect(new URL(`/${locale}${pathname}`, request.url));
  }

  // Security headers
  const response = NextResponse.next();
  response.headers.set('X-Frame-Options', 'DENY');
  response.headers.set('X-Content-Type-Options', 'nosniff');
  return response;
}

export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|public/).*'],
};
```

NX-07: Streaming & Suspense

🇻🇳 Streaming SSR cho phép server gửi HTML từng phần thay vì đợi toàn bộ. `loading.tsx` tạo Suspense boundary tự động cho route segment. Dùng `<Suspense>` trực tiếp trong component để streaming chi tiết hơn. Kết hợp với Server Components giúp TTFB nhanh hơn nhiều.



"Streaming SSR is one of the most impactful features in the App Router. Instead of waiting for all data to resolve before sending any HTML, the server progressively streams parts of the page as they become ready. `loading.tsx` creates an automatic Suspense boundary for its route segment — the shell renders instantly while the data-dependent content streams in. For more granular control, I wrap specific components in `<Suspense>` with custom fallbacks. This dramatically improves Time to First Byte and perceived performance — the user sees the page layout immediately while heavy data like transaction histories load in. For a banking dashboard, this means showing the navigation and account summary first while transaction details stream in."

```
// app/dashboard/loading.tsx – Auto Suspense boundary
export default function DashboardLoading() {
  return <DashboardSkeleton />;
}

// app/dashboard/page.tsx – Granular streaming
import { Suspense } from 'react';

export default function Dashboard() {
  return (
    <div className="grid grid-cols-2 gap-4">
      {/* These stream independently! */}
      <Suspense fallback={<AccountSummarySkeleton />}>
        <AccountSummary />    {/* Fast – streams first */}
      </Suspense>

      <Suspense fallback={<TransactionsSkeleton />}>
        <RecentTransactions />  {/* Slow API – streams when ready */}
      </Suspense>

      <Suspense fallback={<ChartSkeleton />}>
        <SpendingChart />      {/* Heavy computation – streams last */}
      </Suspense>
    </div>
  );
}

// Each component fetches its own data – no waterfall, no blocking
async function AccountSummary() {
  const data = await fetch('https://api.anz.com/summary'); // 200ms
  const summary = await data.json();
  return <div>{summary.balance}</div>;
}
```

NX-08: Image Optimization

🇻🇳 next/image tự động optimize: lazy loading, responsive sizes, format chuyển WebP/AVIF, prevent CLS với width/height bắt buộc. Blur placeholder dùng `placeholder="blur"` với `blurDataURL` (base64 nhỏ ~10 bytes). Remote images cần config `remotePatterns` trong `next.config.js`.



"The next/image component handles image optimization automatically — it lazy loads by default, serves modern formats like WebP and AVIF, and generates responsive srcsets. It requires width and height props which prevents Cumulative Layout Shift — critical for Core Web Vitals. For placeholders, I use `placeholder='blur'` with either a static import that auto-generates the blur hash, or a tiny base64 `blurDataURL` for dynamic images. Remote images require configuring `remotePatterns` in `next.config.js` for security. At ANZ, this is relevant for product imagery, marketing banners, and any customer-facing content where LCP is measured."

```
import Image from 'next/image';
import heroImage from '@public/hero.jpg'; // Static import → auto blur

// Static image with auto blur placeholder
<Image
  src={heroImage}
  alt="ANZ Banking"
  placeholder="blur" // Auto-generated from static import
  sizes="(max-width: 768px) 100vw, 50vw"
  priority // Preload for LCP image
/>

// Dynamic remote image
<Image
  src={`https://cdn.anz.com/products/${id}.jpg`}
  alt={product.name}
  width={800}
  height={600}
  placeholder="blur"
  blurDataURL="..." // Tiny base64
  sizes="(max-width: 640px) 100vw, (max-width: 1024px) 50vw, 33vw"
/>

// next.config.js
module.exports = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'cdn.anz.com', pathname: '/products/**' },
    ],
  },
};
```

NX-09: Metadata API & SEO

🇻🇳 App Router có Metadata API thay thế `next/head` . Export `metadata` object (static) hoặc `generateMetadata` function (dynamic, async). Hỗ trợ OpenGraph, Twitter cards, robots, sitemap, JSON-LD. Metadata tự merge từ layout → page (page overrides layout).



"The Metadata API in App Router replaces `next/head` with a much cleaner approach. For static pages, you export a `metadata` object. For dynamic pages, you use `generateMetadata` — an async function that receives the route params so you can fetch data for dynamic titles and OG images. Metadata cascades from layouts down to pages, with deeper levels overriding parent values. I also generate `sitemap.xml` and `robots.txt` using special files in the app directory. For structured data, I inject JSON-LD scripts in the page component. This is essential for any public-facing ANZ pages where SEO and social sharing matter."

```

// app/layout.tsx – Base metadata (inherited by all pages)
import type { Metadata } from 'next';

export const metadata: Metadata = {
  metadataBase: new URL('https://www.anz.com.au'),
  title: { default: 'ANZ Banking', template: '%s | ANZ' },
  description: 'ANZ Bank Australia',
  openGraph: { type: 'website', locale: 'en_AU', siteName: 'ANZ' },
};

// app/products/[slug]/page.tsx – Dynamic metadata
export async function generateMetadata({ params }: { params: { slug: string } }): Promise<Metadata> {
  const product = await getProduct(params.slug);
  return {
    title: product.name, // Renders: "Home Loan | ANZ"
    description: product.summary,
    openGraph: {
      title: product.name,
      description: product.summary,
      images: [{ url: product.ogImage, width: 1200, height: 630 }],
    },
  };
}

// app/sitemap.ts – Auto-generated sitemap
export default async function sitemap() {
  const products = await getProducts();
  return [
    { url: 'https://www.anz.com.au', lastModified: new Date() },
    ...products.map(p => ({
      url: `https://www.anz.com.au/products/${p.slug}`,
      lastModified: p.updatedAt,
    })),
  ];
}

```

NX-10: Parallel Routes & Intercepting Routes

🚩 **Parallel Routes** (@folder): render nhiều pages cùng lúc trong 1 layout (dashboard có @analytics + @team + @notifications). **Intercepting Routes** ((.) , (..) , (...)): "chặn" navigation để show modal thay vì full page (click ảnh → modal, refresh → full page). Instagram-style pattern.



"Parallel Routes use the @folder convention to render multiple page components simultaneously in a single layout — each slot can have its own loading and error states. Think of a dashboard with @analytics, @team, and @notifications panels that load independently. Intercepting Routes let you 'intercept' a navigation to show a different view — like clicking a photo in a feed opens a modal, but sharing the URL or refreshing shows the full page. This is the Instagram or Twitter pattern. The convention uses (.) for same level, (..) for one level up. At ANZ, parallel routes are great for complex dashboards, and intercepting routes could be used for transaction detail modals."

```
app/  
├─ layout.tsx                # Receives { analytics, team } as props  
├─ @analytics/  
│   ├─ page.tsx              # Analytics panel  
│   └─ loading.tsx           # Independent loading state  
├─ @team/  
│   ├─ page.tsx              # Team panel  
│   └─ error.tsx             # Independent error boundary  
├─ page.tsx                  # Main content  
├─  
├─ feed/  
│   ├─ page.tsx              # Transaction feed  
│   └─ (..)transaction/[id]/  # Intercepting route  
│       └─ page.tsx          # → Shows as modal overlay  
└─ transaction/  
    └─ [id]/  
        └─ page.tsx          # → Full page (direct URL / refresh)
```



```
// app/layout.tsx – Parallel routes as slots
export default function DashboardLayout({
  children,
  analytics,
  team,
}: {
  children: React.ReactNode;
  analytics: React.ReactNode;
  team: React.ReactNode;
}) {
  return (
    <div className="grid grid-cols-3 gap-4">
      <main className="col-span-2">{children}</main>
      <aside>
        {analytics}
        {team}
      </aside>
    </div>
  );
}
```

NX-11: Next.js 14 Caching (4 Layers)

🇺🇸 Next.js 14 có 4 lớp cache: (1) **Request Memoization** — dedupe cùng URL trong 1 render (React tự làm), (2) **Data Cache** — cache fetch responses trên server (persistent, revalidate bằng tag/time), (3) **Full Route Cache** — cache HTML + RSC payload ở build time (static routes), (4) **Router Cache** — cache ở client browser, prefetch results (30s dynamic, 5min static). Hiểu cache layers giúp debug "tại sao data cũ" hiệu quả.



"Next.js 14 has four caching layers and understanding them is crucial for debugging stale data issues. First, Request Memoization — React automatically deduplicates identical fetch calls within a single render pass, so multiple components fetching the same URL only trigger one request. Second, the Data Cache — fetch responses are persisted on the server across requests until explicitly revalidated with `revalidateTag` or time-based ISR. Third, the Full Route Cache — statically rendered routes are cached as HTML and RSC Payload at build time. Fourth, the Router Cache on the client — prefetched and visited routes are cached in the browser for 30

seconds for dynamic and 5 minutes for static segments. At ANZ, I'd use `revalidateTag` for transactional data and make sure sensitive financial data uses `cache: 'no-store'` to bypass the Data Cache entirely."

```
// Layer 1: Request Memoization (auto, same render pass)
// Both components fetch same URL → only 1 actual request
async function Header() {
  const user = await fetch('/api/user'); // Request #1
  return <h1>{user.name}</h1>;
}
async function Sidebar() {
  const user = await fetch('/api/user'); // Deduped! Same as #1
  return <p>{user.email}</p>;
}

// Layer 2: Data Cache (persistent, revalidate with tags)
const data = await fetch('https://api.anz.com/rates', {
  next: { tags: ['exchange-rates'], revalidate: 3600 } // Cache 1hr, tag for on-demand
});

// On-demand revalidation (Server Action or Route Handler)
import { revalidateTag } from 'next/cache';
revalidateTag('exchange-rates'); // Bust Data Cache for this tag

// Layer 3: Full Route Cache → controlled by dynamic functions
export const dynamic = 'force-dynamic'; // Opt out of Full Route Cache
export const revalidate = 60; // Or ISR with time-based revalidation

// Layer 4: Router Cache → client-side, controlled by:
import { useRouter } from 'next/navigation';
const router = useRouter();
router.refresh(); // Invalidate Router Cache for current route
```

NX-12: ISR — Incremental Static Regeneration

★ ISR = static page + revalidation. Time-based: `revalidate: N` (serve stale, rebuild background). On-demand: `revalidatePath()` / `revalidateTag()` gọi từ webhook/Server Action khi data thay đổi. Next 14 dùng cả trong App Router lẫn Pages Router. Phù hợp cho: product pages, blog, exchange rates — content thay đổi không liên tục.



"ISR gives you the performance of static pages with the freshness of dynamic data. Time-based ISR serves the cached page and rebuilds it in the background after the revalidation period expires — the next visitor gets the fresh version. On-demand ISR is more precise — you call `revalidatePath` or `revalidateTag` from a webhook or Server Action when data actually changes, instead of polling. In the App Router, you set revalidation at the fetch level or at the route segment level with `export const revalidate = N`. For ANZ, I'd use time-based ISR for exchange rate pages that update every few minutes, and on-demand revalidation triggered by a CMS webhook for marketing content pages."

```
// Time-based ISR at route segment level
// app/rates/page.tsx
export const revalidate = 300; // Rebuild every 5 minutes

export default async function ExchangeRates() {
  const rates = await fetch('https://api.anz.com/exchange-rates');
  return <RatesTable data={await rates.json()} />;
}

// On-demand ISR via webhook
// app/api/revalidate/route.ts
import { revalidatePath, revalidateTag } from 'next/cache';
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  const secret = request.headers.get('x-revalidate-secret');
  if (secret !== process.env.REVALIDATION_SECRET) {
    return NextResponse.json({ error: 'Invalid secret' }, { status: 401 });
  }

  const { type, slug } = await request.json();

  if (type === 'product') {
    revalidateTag('products'); // Revalidate all tagged fetches
    revalidatePath(`/products/${slug}`); // Revalidate specific page
  }

  return NextResponse.json({ revalidated: true, now: Date.now() });
}
```

QUICK REFERENCE CARDS

React Hooks Cheat Sheet

Hook	Purpose	Key Point
useState	Local state	Functional update for prev-dependent state
useEffect	Side effects	Cleanup before next effect & unmount
useCallback	Cache function ref	Only useful with React.memo children
useMemo	Cache computed value	Only for expensive calculations
useRef	Mutable value, DOM	Doesn't trigger re-render
useContext	Consume context	Every consumer re-renders on change
useReducer	Complex state	Centralized transitions, testable

TypeScript Utility Types

Type	What it does	Use case
Partial<T>	All fields optional	Update DTOs
Required<T>	All fields required	Strict validation
Pick<T,K>	Select specific fields	API response subsets
Omit<T,K>	Remove specific fields	Hide internal fields
Record<K,V>	Object with typed keys/values	Lookup tables
ReturnType<T>	Function return type	Infer from functions
Readonly<T>	All fields readonly	Immutable state

Event Loop Order

1. Synchronous code (Call Stack)
 2. Microtasks (Promise.then, queueMicrotask)
 3. Macrotasks (setTimeout, setInterval, I/O)
- After EACH macrotask, ALL microtasks run

RADIO Framework

- R – Requirements: Functional + Non-functional
- A – Architecture: Components + State + Services
- D – Data Model: Interfaces + API contracts + Caching
- I – Interface: Props + Events + Composition
- O – Optimizations: Performance + Allly + Edge cases

Core Web Vitals

Metric	Measures	Good
LCP	Loading	< 2.5s
CLS	Visual stability	< 0.1
INP	Responsiveness	< 200ms

Good luck with the interview! 💪